Numerically stable gradients of Golub-Kahan Bidiagonalization

Theo Würtzen and Nicholas Krämer

September 9, 2025

Contents

1 Rambling:

Bidiagonalization suffers from loss of orthogonality of the vectors in L and R. This can be solved by reorthonormalizing the new vector against all computed vectors.

Ideally, that should've been enough, but it can be shown that it is not - we need at least 3 applications of reorthonormalization for this to work. [1].

In the adjoint system we can do something similar.

2 Prerequisites

2.1 Matrix free linear algebra

Matrix-free linear algebra refers to implementing matrix operations without explicitly forming or storing the full matrices. Instead, we work with functions that compute matrix-vector products. This is particularly useful when dealing with large matrices where storing them would be impractical, or when the matrix is defined implicitly through some operation, where we never need the full matrix, only specific matrix-vector products. For a linear operator $A : \mathbb{R}^m \to \mathbb{R}^n$, instead of storing an $n \times m$ matrix, we represent it as a function:

$$\text{matvec} : \mathbb{R}^m \times \Theta \to \mathbb{R}^n$$

such that $\text{matvec}(v, \theta) = A_\theta v$ for any vector $v \in \mathbb{R}^m$ and parameters $\theta \in \Theta$. This approach enables a general way to handle general linear operators, such as matrix-vector products with sparse and structured matrices, the jacobian-vector product of a neural network, finite-difference operators, discrete fourier transforms and other linear operators acting between vector spaces.

2.2 Hessenberg Factorization

$$\texttt{Hessenberg} : (A, v) \mapsto (Q, H, r, c)$$

Let e_j be the jth unit vector. Denote by "$\circ$" the element-wise matrix product, and define the matrices

$$I_{\leq} := [\delta_{i \leq j}]_{i,j=1}^{K} \qquad I_{<} := [\delta_{i<j}]_{i,j=1}^{K} \qquad I_{\ll} := [\delta_{i+1<j}]_{i,j=1}^{K}$$

so that for example, $I_{\leq} \circ A$ extracts the lower triangular matrix of A (including the diagonal), and $I_{\ll} \circ A = 0$ enforces Hessenberg form.

Given a square matrix $A \in \mathbb{R}^{N \times N}$ an initial vector $v \in \mathbb{R}^N$ and a prescribed number of iterations $K \in \mathbb{N}_+$, the Hessenberg factorization produces a column-orthogonal $Q \in \mathbb{R}^{N \times K}$,

[1] Show this in an experiment

structured $H \in \mathbb{R}^{K\times K}$, residual vector $r \in \mathbb{R}^N$, and length $c \in \mathbb{R}$ such that

$$AQ = QH + r(e_K)^\top \tag{1}$$
$$Qe_1 = cv \tag{2}$$
$$I_{\leq} \circ [Q^\top Q] = I \tag{3}$$
$$I_{\ll} \circ H = 0 \tag{4}$$
$$Q^\top r = 0 \tag{5}$$

2.3 Bidiagonalization

$$\texttt{Bidiagonalize} : (A, v) \mapsto (L, R, B, r, c)$$

Bidiagonalizing an arbitrary matrix $A \in \mathbb{R}^{N\times M}$ with initial vector $v \in \mathbb{R}^M$ and a prescribed number of iterations $K \in \mathbb{N}_+$, will yield column-orthonormal $L \in \mathbb{R}^{N\times K}$ and $R \in \mathbb{R}^{M\times K}$, structured $B \in \mathbb{R}^{K\times K}$, residual vector $r \in \mathbb{R}^N$ and length $c \in \mathbb{R}$, satisfying the following constraints:

$$A^\top L = RB^\top + r(e_k)^\top \tag{6}$$
$$AR = LB \tag{7}$$
$$Re_1 = cv \tag{8}$$
$$I_{\leq} \circ [L^\top L] = I \tag{9}$$
$$I_{\leq} \circ [R^\top R] = I \tag{10}$$
$$R^\top r = 0 \tag{11}$$

and B is diagonally banded as follows:

$$B = \underbrace{\operatorname{diag}(\boldsymbol{\alpha}) + \operatorname{diag}(\boldsymbol{\beta}, \text{k=1})}_{\text{B is 'upper bidiagonal'}} = \begin{bmatrix} \alpha_1 & \beta_1 & 0 & 0 & 0 & 0 \\ 0 & \alpha_2 & \beta_2 & 0 & 0 & 0 \\ 0 & 0 & \ddots & \ddots & 0 & 0 \\ 0 & 0 & 0 & 0 & \alpha_{K-1} & \beta_{K-1} \\ 0 & 0 & 0 & 0 & 0 & \alpha_K \end{bmatrix} \in \mathbb{R}^{K\times K} \tag{12}$$

2.4 Reprojection and Reorthonormalization

Let m be a mask vector with elements in $\{0, 1\}$ and $\circ$ denoting the element-wise product. Let L be an orthonormal matrix, and q and c be vectors. We are given a constraint of the form

$$m \circ (L^\top q) = m \circ c$$

The constraint states: "Within the bounds of the mask m, we have that q expressed in the basis of L equals c". Given a numerically contaminated estimate q_0 of a solution q, we can

improve our estimate of q by *reprojecting* it against L. The following motivates following step, which can be repeated if necessary.

$$\begin{aligned} q_0 &= q_0 - LL^\top q_0 + LL^\top q_0 \\ &= (I - LL^\top)q_0 + Lc \\ &=: q_1 \end{aligned}$$

In the case of Hessenberg factorization, we have the constraint $Q^\top q_i = e_i$ in which case we call the reprojection step followed by normalization *reorthonormalization*.

3 Bidiagonalization through Hessenberg factorization

While Bidiagonalization has well-understood forward algorithms and constraints, computing its adjoint (gradient) in a numerically stable way is challenging (TODO show that it is unstable from example). However, Hessenberg factorization has known stable adjoint methods (TODO cite Nico). This paper builds from a connection between Bidiagonalization and Hessenberg factorization, namely that any Bidiagonalization can be computed by performing Hessenberg factorization on an appropriately constructed symmetric matrix, allowing us to then leverage the stable adjoint algorithms of Hessenberg factorization proposed in (TODO cite Nico).

To compute $\texttt{Bidiagonalize}(A, v) = (L, R, B, r, c)$ for any $A \in \mathbb{R}^{N\times M}$ and $v \in \mathbb{R}^M$ using Hessenberg factorization, we construct the symmetric matrix $\bar{A}$ and augmented vector $\bar{v}$:

$$\bar{A} := \begin{bmatrix} \mathbf{0} & A \\ A^\top & \mathbf{0} \end{bmatrix} \in \mathbb{R}^{(N+M)\times(N+M)} \quad \text{and} \quad \bar{v} := \begin{pmatrix} \mathbf{0} \\ v \end{pmatrix} \in \mathbb{R}^{N+M}$$

We define the operation $\texttt{augment} : (A, v) \mapsto (\bar{A}, \bar{v})$ as the augmentation of a matrix and vector to this symmetric matrix and padded vector. Applying Hessenberg factorization to this constructed input yields

$$\texttt{Hessenberg}(\bar{A}, \bar{v}) = (\bar{Q}, \bar{H}, \bar{r}, c)$$

which are directly related to (L, R, B, r, c) through the following sparse equations:

$$\bar{Q} = \begin{bmatrix} \mathbf{0} & l_1 & \mathbf{0} & l_2 & \cdots & \mathbf{0} & l_k \\ r_1 & \mathbf{0} & r_2 & \mathbf{0} & \cdots & r_k & \mathbf{0} \end{bmatrix} \in \mathbb{R}^{(N+M)\times K} \qquad \bar{r} := \begin{pmatrix} \mathbf{0} \\ r \end{pmatrix}$$

$$\bar{H} = \begin{bmatrix} 0 & \alpha_1 & 0 & 0 & 0 & 0 & 0 & 0 \\ \alpha_1 & 0 & \beta_1 & 0 & 0 & 0 & 0 & 0 \\ 0 & \beta_1 & 0 & \alpha_2 & 0 & 0 & 0 & 0 \\ 0 & 0 & \alpha_2 & 0 & \beta_2 & 0 & 0 & 0 \\ 0 & 0 & 0 & \beta_2 & 0 & \ddots & 0 & 0 \\ 0 & 0 & 0 & 0 & \ddots & 0 & \beta_{K-1} & 0 \\ 0 & 0 & 0 & 0 & 0 & \beta_{K-1} & 0 & \alpha_K \\ 0 & 0 & 0 & 0 & 0 & 0 & \alpha_K & 0 \end{bmatrix} \in \mathbb{R}^{2K\times 2K}$$

The vectors l_i and r_i in $\bar{Q}$ are exactly the columns of L and R, while the bands in $\bar{H}$ hold the entries of the bidiagonal matrix B with duplicated α_i and β_i values. The residual vector $\bar{r}$ is simply a padded version of r, and the inverse norm c is unchanged.

We formalize this relationship through an extraction mapping $\texttt{extract} : (\bar{Q}, \bar{H}, \bar{r}, c) \mapsto (L, R, B, r, c)$ that projects away the zeros and extracts the relevant non-zero entries from the Hessenberg outputs.[2] This allows us to define **augmented Bidiagonalization** as $\overline{\texttt{Bidiagonalize}} = \texttt{extract} \circ \texttt{Hessenberg} \circ \texttt{augment}$, which is functionally equivalent to standard Bidiagonalization but inherits the numerical stability properties of Hessenberg factorization's adjoint system. This equivalence is the foundation for our stable gradient computation approach in the following sections. This composition however has an computational overhead vs just calling Bidiagonalization directly, so the remaining section will focus on pruning the computational graph to minimize work.

4 Adjoint systems

4.1 The Adjoint Method to compute gradients

The adjoint method provides an elegant way to compute parameter gradients of losses that depend on outputs of parameterized numerical solvers.

Let $s : \theta \mapsto x$ be a numerical solver that yields a solution x to some problem θ, with $\dim(\theta) = P$ and $\dim(x) = X$. Assume also that we know an associated constraint function $c : x \mapsto \mathbb{R}^X$ that evaluates all the constraints of the problem, for which $c(s(\theta), \theta) = \mathbf{0}$ for all θ. Additionally, require that $\frac{\partial c}{\partial \theta}$ is everywhere nonsingular. Assume also that we have a loss function $\ell : x \mapsto \mathbb{R}$ that depends on the solution x of the problem. We are interested in $\nabla_\theta \ell(s(\theta))$, the gradient of ℓ with respect to the parameters θ.

Example 1 (Hessenberg Factorization)**.** A fitting example would be s = Hessenberg-factorization, which implies that $\theta = (A, v)$ and $x = (Q, H, r, c)$. The associated constraint function would be the concatenation of (1) through (5). We see that, for this example, $c(x, \theta) = \mathbf{0}$ for all solutions (Q, H, r, c) if and only if they correctly Hessenberg-factorize from $\theta = (A, v)$.

Because $c(s(\theta), \theta) = \mathbf{0}$ is constantly zero, the total derivative $\frac{d}{d\theta} c$ is also zero. With the chain rule, we have:

$$\mathbf{0} = \frac{d}{d\theta} c(s(\theta), \theta) = \frac{\partial c}{\partial s}\frac{\partial s}{\partial \theta} + \frac{\partial c}{\partial \theta} \implies \frac{\partial c}{\partial s}\frac{\partial s}{\partial \theta} = -\frac{\partial c}{\partial \theta} \tag{13}$$

[2]The projection from $\bar{H}$ to B is not unique, as $\bar{H}$ contains two copies of each α_i and β_i. We arbitrarily select the superdiagonal entries and drop the subdiagonal entries. This choice affects the sparsity pattern of $\nabla_{\bar{H}} \rho$ in section 4.7.

The adjoint method will exploit this relationship. We apply the chain rule to compute the gradient $\nabla_\theta \ell(s(\theta))$:

$$\nabla_\theta \ell(s(\theta)) = \frac{\partial s}{\partial \theta}^\top \nabla_x \ell(s(\theta)) \tag{14}$$

We assume that $\nabla_x \ell(s(\theta))$ is easy to obtain. The challenge is computing $\frac{\partial s}{\partial \theta} \in \mathbb{R}^{X \times P}$, which can be expensive for high-dimensional x. Suppose we can find *adjoint variables* $\boldsymbol{\lambda} \in \mathbb{R}^X$ solving the uniquely determined $X \times X$ linear system (unique by initial assumption on the constraint Jacobian)

$$\nabla_x \ell(s(\theta)) = \frac{\partial c}{\partial s}^\top \boldsymbol{\lambda} \tag{15}$$

Then, substituting this into (14) yields:

$$\nabla_\theta \ell(s(\theta)) = \frac{\partial s}{\partial \theta}^\top \frac{\partial c}{\partial s}^\top \boldsymbol{\lambda} = \left(\frac{\partial c}{\partial s}\frac{\partial s}{\partial \theta}\right)^\top \boldsymbol{\lambda} \overset{(13)}{=} -\frac{\partial c}{\partial \theta}^\top \boldsymbol{\lambda}$$

The adjoint method provides significant computational savings compared to the standard approach. To see this, consider the two approaches:

Forward approach: Compute $\frac{\partial s}{\partial \theta} \in \mathbb{R}^{X \times P}$ directly by solving P linear systems of size $X \times X$ (one for each parameter), then compute the gradient via (14). Total cost: $O(PX^3)$ operations.

Adjoint approach: Solve one linear system (15) of size $X \times X$ to find $\boldsymbol{\lambda}$, then evaluate $-\frac{\partial c}{\partial \theta}^\top \boldsymbol{\lambda}$. Total cost: $O(X^3 + XP)$ operations.

When $P \ll X$ (few parameters, high-dimensional state), the adjoint method is dramatically more efficient: $O(X^3)$ versus $O(PX^3)$. This advantage becomes crucial in applications like neural network training or PDE-constrained optimization where X can be very large.

4.2 Adjoint system of Hessenberg Factorization

Let $K \in \mathbb{N}_+$, $A \in \mathbb{R}^{N \times N}$, $v \in \mathbb{R}^N$, $H \in \mathbb{R}^{K \times K}$, $Q \in \mathbb{R}^{N \times K}$ and $r \in \mathbb{R}^N$. Let adjoint variables $\Lambda \in \mathbb{R}^{N \times K}$, $\Gamma \in \mathbb{R}^{K \times K}$, $\lambda \in \mathbb{R}^N$, $\gamma \in \mathbb{R}^K$ solve the following system of equations:

$$0 = \nabla_Q \rho + A^\top \Lambda - \Lambda H^\top + \lambda (e_1)^\top + Q(I_{\leq} \circ \Gamma) + Q(I_{\leq} \circ \Gamma)^\top + r\gamma^\top \tag{16}$$

$$0 = \nabla_H \rho - Q^\top \Lambda + I_{\leq} \circ \Sigma \tag{17}$$

$$0 = \nabla_r \rho - \Lambda e_K + Q\gamma \tag{18}$$

$$0 = \nabla_c \rho - v^\top \lambda \tag{19}$$

The gradients $\nabla_A \rho$ and $\nabla_v \rho$ are then given by[3]:

$$\nabla_A \rho = \Lambda Q^\top \tag{20}$$

$$\nabla_v \rho = -c\lambda \tag{21}$$

[3]note a sign difference in $\nabla_v \rho$ due to a typo

4.3 Hessenberg factorization has stable adjoint because of reprojection

Some of the constraints of the adjoint system in Hessenberg factorization allow for a direct correction of drift [4]. Because Hessenberg factorization subsumes Bidiagonalization for this specific structured input, we can use their implementation of numerically stable gradients to differentiate bidiagonalization.

4.4 Unstable Primal constraints

Define $l_0 b_0 := 0$ and $res := r_{k+1} b_k$

$$\begin{aligned} r_1 &= c\tilde{r} \\ \alpha_n l_n &= A r_n - \beta_{n-1} l_{n-1} \\ \beta_n r_{n+1} &= A^\top l_n - \alpha_n r_n \\ \langle l_i, l_i \rangle &= 1 \text{ for } i \in 1 \dots k \\ \langle r_i, r_i \rangle &= 1 \text{ for } i \in 1 \dots k \end{aligned}$$

Symbolically, we can[5] show that these constraints, if satisfied, imply $\langle l_i, l_j \rangle = \delta_{i,j}$ and $\langle r_i, r_j \rangle = \delta_{i,j}$. In practice however, we need to reorthonormalize the produced vectors during iteration.
These "classic" forward constraints and the associated adjoint system:

4.5 Unstable Adjoint system

[6]

$$\rho_k = -\nabla res \tag{22}$$
$$c\nabla c = r_1^\top \kappa \tag{23}$$
$$\nabla \tilde{r} = -c\kappa \tag{24}$$
$$0 = \nabla L - AP + \Lambda B^\top + L\Sigma \tag{25}$$
$$0 = \nabla R - A^\top \Lambda + PB + R\Omega + \delta_{i,1}\kappa \tag{26}$$
$$\langle l_n, \lambda_n \rangle + \langle r_n, \rho_n \rangle = \nabla a_n \text{ for } i \in \{1 \cdots k\} \tag{27}$$
$$\langle l_n, \lambda_{n+1} \rangle + \langle r_{n+1}, \rho_n \rangle = \nabla b_n \text{ for } i \in \{1 \cdots k-1\} \tag{28}$$
$$\nabla A = \sum_{n=1}^{k} -\lambda_n r_n^\top - l_n \rho_n^\top \tag{29}$$

[4] cite equation 13b
[5] proof by induction. Add?
[6] See another subsection for the same things all expanded columnwise

4.6 Stable Adjoint System

$$\downarrow_k = \nabla_{res} + R\gamma \tag{30}$$
$$r_1^\top \kappa = c\nabla_c \tag{31}$$
$$0 = \nabla_L + A \Downarrow - \Uparrow B^\top + L(\Sigma + \Sigma^\top) \tag{32}$$
$$0 = \nabla_R + A^\top \Uparrow - \Downarrow B + R(\Omega + \Omega^\top) + \kappa e_1^\top + res\gamma^\top \tag{33}$$
$$\langle l_i, \uparrow_j \rangle = \nabla \beta_{i-1} e_{i-1} \text{ for } i \leq j \tag{34}$$
$$\langle r_i, \downarrow_j \rangle = \nabla \alpha_i e_i \text{ for } i \leq j+1 \tag{35}$$
$$\nabla_v = -\kappa c \tag{36}$$
$$\nabla_A = \sum_{i=1}^{k} l_i \downarrow_i^\top + \uparrow_i r_i^\top \tag{37}$$

4.7 Derivation of the Stable Adjoint System

If we restrict the inputs to Hessenberg factorization to matrices $\bar{A}$ and vectors $\bar{v}$, we get the relationship between Hessenberg Factorization and Bidiagonalization as described in 3. We can then further combine this with the adjoint system of Hessenberg Factorization as described in 4.2. The idea is to prune away unnecessary computation and avoid "creating the zeros". We will look at the constraints (16) through (19) one at a time, and see how they can be simplified by using the relationship between Hessenberg Factorization and Bidiagonalization.

4.7.1 Constraint (16)

$$0 = \nabla_Q \rho + A^\top \Lambda - \Lambda H^\top + \lambda (e_1)^\top + Q(I_{\leq} \circ \Gamma) + Q(I_{\leq} \circ \Gamma)^\top + r\gamma^\top \tag{38}$$

It is most easily shown by choosing a small $k = 3$ and seeing the concrete pattern. The sparsity patterns of $\nabla_Q \rho$ and $\nabla_H \rho$ are the following, due to the way we

$$\nabla_Q \rho = \begin{bmatrix} \mathbf{0} & \nabla_{l_1}\rho & \mathbf{0} & \nabla_{l_2}\rho & \cdots & \mathbf{0} & \nabla_{l_k}\rho \\ \nabla_{r_1}\rho & \mathbf{0} & \nabla_{r_2}\rho & \mathbf{0} & \cdots & \nabla_{r_k}\rho & \mathbf{0} \end{bmatrix} \in \mathbb{R}^{(N+M)\times K}$$

4.8 Columnwise expanded unstable adjoint

Define $b_n\lambda_{n+1} := \mathbf{0}$ and $b_0\rho_0 := \mathbf{0}$

$$0 = \nabla l_n + a_n\lambda_n + b_n\lambda_{n+1} - A\rho_n + l_n\sigma_n \tag{39}$$

$$0 = \nabla r_n - A^\top\lambda_n + b_{n-1}\rho_{n-1} + a_n\rho_n + r_n\omega_n + \delta_{i,1}\kappa \tag{40}$$

$$\rho_k = -\nabla res \tag{41}$$

$$\nabla a_n + l_n^\top\lambda_n + r_n^\top\rho_n = 0 \text{ for } i \in \{1\cdots k\} \tag{42}$$

$$\nabla b_n + l_n^\top\lambda_{n+1} + r_{n+1}^\top\rho_n = 0 \text{ for } i \in \{1\cdots k-1\} \tag{43}$$

$$\nabla c - \tilde{r}^\top\kappa = 0 \tag{44}$$

$$\nabla\tilde{r} = -c\kappa \tag{45}$$

$$\nabla A = \sum_{n=1}^{k} -\lambda_n r_n^\top - l_n\rho_n^\top \tag{46}$$